

colab.research.google.com/drive/1_JbZgQ0nA5aTa5sHegFxnLw1uI0JkO0z#scrollTo=vvTDDrHDjE9t

G.231.24.0001_Rikabaguswijaya_Prak3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

```
Nama:Rika bagus wijaya
Nim : G.231.24.0001
Matkul : Analis kompleksitas algoritma
Praktikum 3

PSEUDOCODE

Mulai
Input jumlah node
Input graph
Input titik awal
Set semua jarak = tak hingga
Set jarak titik awal = 0
```

Variables Terminal 12:00 AM Python 3

colab.research.google.com/drive/1_JbZgQ0nA5aTa5sHegFxnLw1uI0JkO0z#scrollTo=vvTDDrHDjE9t

G.231.24.0001_Rikabaguswijaya_Prak3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

```
Selama masih ada node yang belum dikunjungi

Pilih node dengan jarak terkecil
Tandai node sebagai sudah dikunjungi

Untuk setiap tetangga node
    Hitung jarak baru

    Jika jarak baru lebih kecil
        Update jarak
    End If
End For

End While
Tampilkan jarak terpendek
Selesai
```

Variables Terminal 12:00 AM Python 3

G231252-2167: Materi Praktiku...G.231.24.0001_Rikabaguswijaya

colab.research.google.com/drive/1_JbZgQ0nA5aTa5sHegFxnLw1u0JkO0z#scrollTo=vvTDDrHDjE9t

G.231.24.0001_Rikabaguswijaya_Prak3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Code

```
code = r'''
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

const int INF = 999999;

void dijkstra(vector<vector<pair<int,int>>> &graph, int start)
{
    int n = graph.size();
    vector<int> dist(n, INF);

    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

    dist[start] = 0;
    pq.push({0, start});

    while(!pq.empty())
```

Variables Terminal

12:00 AM Python 3

Search

00:07 09/05/2026

G231252-2167: Materi Praktiku...G.231.24.0001_Rikabaguswijaya

colab.research.google.com/drive/1_JbZgQ0nA5aTa5sHegFxnLw1u0JkO0z#scrollTo=vvTDDrHDjE9t

G.231.24.0001_Rikabaguswijaya_Prak3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Code

```
{
    int u = pq.top().second;
    pq.pop();

    for(auto edge : graph[u])
    {
        int v = edge.first;
        int weight = edge.second;

        if(dist[u] + weight < dist[v])
        {
            dist[v] = dist[u] + weight;
            pq.push({dist[v], v});
        }
    }

    cout << "Jarak terpendek dari node awal:" << endl;

    for(int i = 0; i < n; i++)
    {
        cout << "Node " << i << " = " << dist[i] << endl;
    }
}
```

Variables Terminal

12:00 AM Python 3

Search

00:08 09/05/2026

colab.research.google.com/drive/1_JbZgQ0nA5aTa5sHegFxnLw1u0JkO0z#scrollTo=vvTDDrHDjE9t

G.231.24.0001_Rikabaguswijaya_Prak3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[5] ✓ 1s
}

int main()
{
    int n = 5;

    vector<vector<pair<int,int>>> graph(n);

    graph[0].push_back({1, 4});
    graph[0].push_back({2, 1});

    graph[2].push_back({1, 2});
    graph[1].push_back({3, 1});

    graph[2].push_back({3, 5});
    graph[3].push_back({4, 3});

    int start = 0;

    dijkstra(graph, start);
}
```

Variables Terminal

12:00 AM Python 3

00:08 09/05/2026

colab.research.google.com/drive/1_JbZgQ0nA5aTa5sHegFxnLw1u0JkO0z#scrollTo=vvTDDrHDjE9t

G.231.24.0001_Rikabaguswijaya_Prak3.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[5] ✓ 1s
    return 0;
}
...

with open('main.cpp', 'w') as f:
    f.write(code)

!g++ main.cpp -o main
!./main
```

Jarak terpendek dari node awal:

```
Node 0 = 0
Node 1 = 3
Node 2 = 1
Node 3 = 4
Node 4 = 7
```

[2] ✓ 3s

```
!apt-get install g++

... Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
g++ is already the newest version (4:11.2.0ubuntu1).
```

Variables Terminal

12:00 AM Python 3

00:08 09/05/2026

Reading state information... Done
g++ is already the newest version (4:11.2.0-1ubuntu1).
g++ set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.

ANALISA KOMPLEKSITAS WAKTU

Algoritma Dijkstra digunakan untuk mencari jalur terpendek pada graph berbobot.

Kompleksitas waktu Algoritma Dijkstra adalah:

$$[O((V + E) V)]$$

Keterangan:

- (V) = jumlah vertex/node
- (E) = jumlah edge/sisi

Priority Queue digunakan untuk mempercepat proses pencarian node dengan jarak terkecil. Semakin banyak node dan edge, maka waktu eksekusi program juga akan semakin besar.

KESIMPULAN

Algoritma Dijkstra dapat digunakan untuk mencari jarak terpendek dari satu node ke node lainnya.

Dengan menggunakan Priority Queue, proses pencarian menjadi lebih efisien. Metode ini banyak digunakan pada sistem navigasi, pencarian rute, dan jaringan komputer.

Link Google Collab :

https://colab.research.google.com/drive/1_JbZgQ0nA5aTa5sHegFxnLw1ul0JkO0z?usp=sharing